

embarcadero®

EMBARCADERO CONFERENCE 2025

A JORNADA DO HERÓI | 30 ANOS DE
INOVAÇÃO



Delphi Multithreading ECON25

Desbloqueie o Potencial Multi-Core do seu Delphi
Da Complexidade da TThread à Maestria da PPL



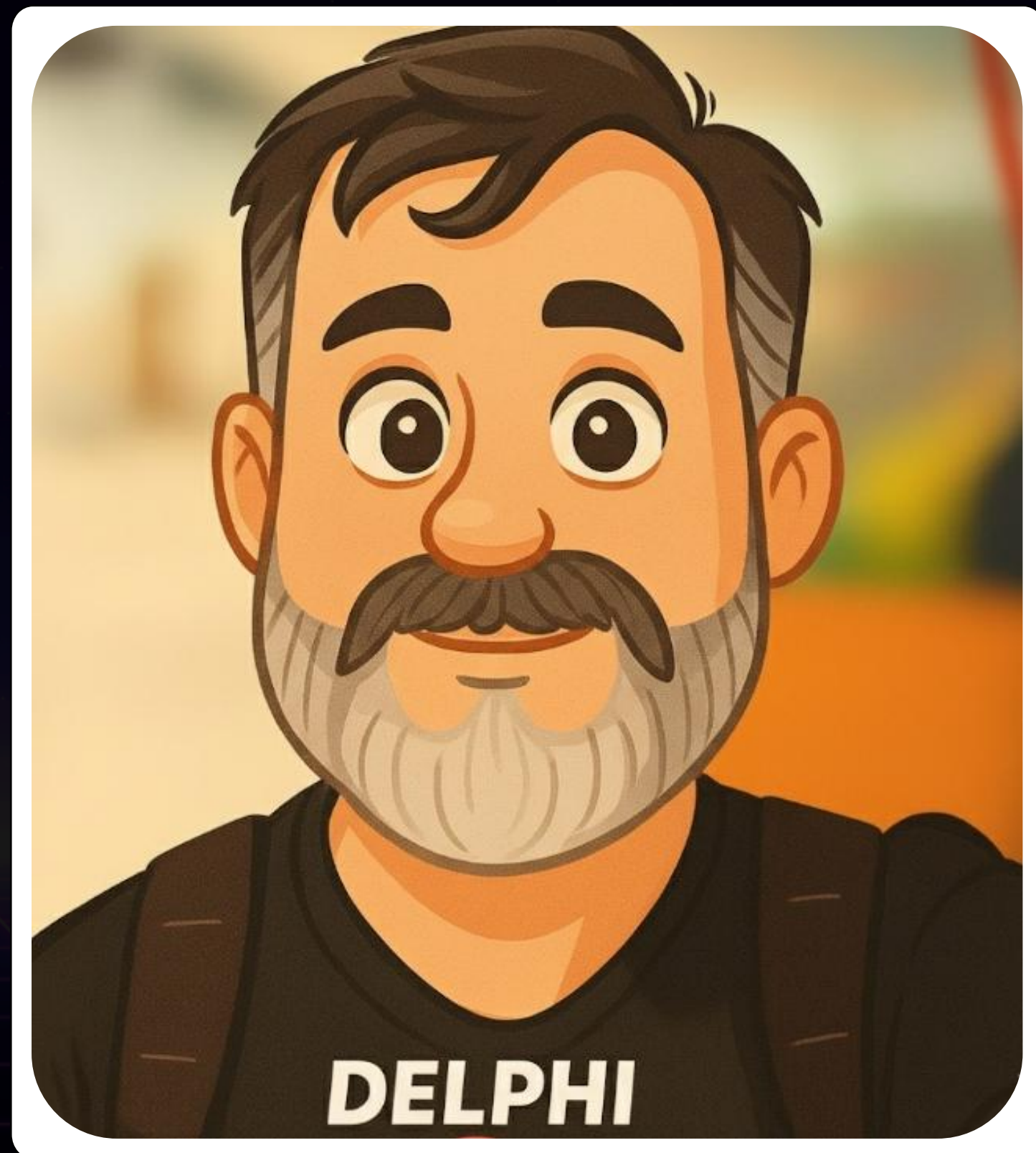
Cesar Romero

30 anos de experiência, Delphi desde 1996.

Carreira focada na construção de sistemas robustos e de alta performance, atuando em projetos complexos envolvendo Desktop, Cloud, Mobile e Micro serviços.

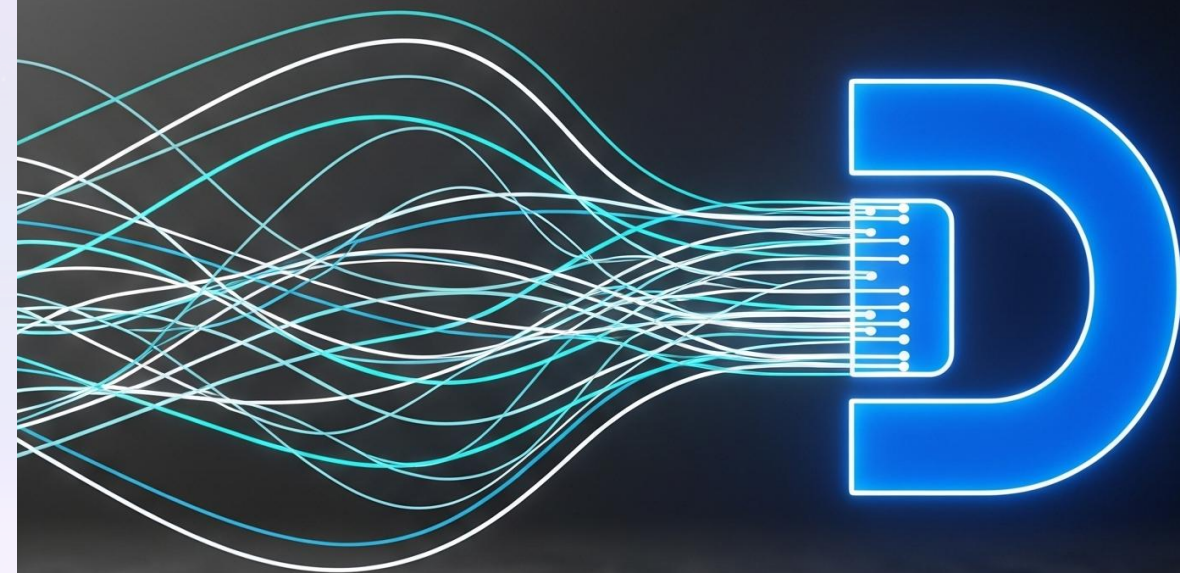
Palestrante internacional, compartilha ativamente seu conhecimento com a comunidade de desenvolvedores.

Autor do livro "Delphi Multithreading: Threads, Concorrência, Paralelismo e Assincronismo".



Delphi Multithreading

Threads, Concorrência, Paralelismo e Assincronismo



Cesar Romero



www.cesarromero.com.br



www.cesarromero.com.br



Concorrência

- Lidar com múltiplas tarefas ao mesmo tempo.
- As tarefas progridem de forma intercalada.
- Não significa execução simultânea.
- **O foco é gerir o que está a acontecer.**





Paralelismo

- Executar múltiplas tarefas simultaneamente.
- Requer múltiplos recursos (núcleos de CPU).
- É uma forma de alcançar a concorrência.
- O foco é fazer mais em menos tempo.





Assincronismo

- Iniciar uma tarefa e não esperar pela sua conclusão.
- O resultado é entregue através de uma notificação ou callback.
- A aplicação permanece responsiva.
- O foco é não bloquear.



TThread (A Ferramenta Clássica)

- É um wrapper de baixo nível para as threads do sistema operacional.
- O gerenciamento do ciclo de vida é manual (Create, WaitFor, Free).
- A comunicação com a UI é explícita e manual, via Synchronize e Queue.
- A sincronização de dados compartilhados exige o uso manual de primitivas como TCriticalSection .
- Cria uma nova thread do sistema a cada instância, o que gera um custo de performance (overhead) .



PPL (A Maestria Moderna)

- É uma abstração de alto nível, baseada em Tarefas, não em threads .
- Possui gerenciamento automático de um Pool de Threads, que reutiliza threads para máxima eficiência.
- Simplifica drasticamente o código com estruturas como **TTask**, **TFuture<T>** e **TParallel.For** .
- Reduz a complexidade e o risco de erros comuns de concorrência.
- O **foco do desenvolvedor** é no "o que" fazer, enquanto a **PPL cuida do "como"**.




```
ClientApp.ParallelFor.Strategy x ClientApp.MainForm
TParallelForStrategy.Execute(const Ids: TArray<Integer>; const StatusCallback: TStatusUpdateCallback; const CompletionCallback: TCompletionEvent);

procedure TParallelForStrategy.Execute(const Ids: TArray<Integer>;
const StatusCallback: TStatusUpdateCallback;
const CompletionCallback: TCompletionEvent);
var
  StrategyIntf: IPipelineStrategy;
begin
  StrategyIntf := Self;

  FParentTask := TTask.Run(
    procedure
    begin
      try
        TParallel.For(0, High(Ids),
          procedure(i: Integer; LoopState: TParallel.TLoopState)
          var
            id: Integer;
            CurrentTask: ITask;
            DownloadResult: TResult<string>;
            FinalResult: TResult<TUserModel>;
          begin
            id := Ids[i];
            try
              CurrentTask := FParentTask;
              if (CurrentTask.Status = TTaskStatus.Canceled) then
              begin
                LoopState.Stop;
                Exit;
              end;
              TThread.Synchronize(nil,
                procedure
                begin
```

Talk is cheap, show me the code!



O que Falta na TThread?



Devo usar TThread ou PPL?

Característica	TThread (Abordagem Clássica)	PPL (Abordagem Moderna)
 Complexidade	Alta O código tende a ser mais verboso e complexo.	Baixa Abstrações que simplificam o código e a lógica.
 Gerenciamento	Manual Requer criação e destruição explícita das threads.	Automático Gerenciado por um <i>Pool de Threads</i> inteligente.
 Risco de Erros	Alto Maior exposição a <i>Deadlocks</i> e <i>Race Conditions</i> .	Reduzido As abstrações de alto nível minimizam erros comuns.
 Performance	Boa Eficiente, mas com custo (<i>overhead</i>) na criação de cada thread.	Otimizada Focada em <i>Multi-core</i> e reutilização de threads.

A Jornada Continua...



A Jornada Continua: O Futuro da PPL

```
· procedure TMainForm.RequestUserInfo(UserId: Integer);  
30 begin  
·   var Task := TTask.Async<string>(  
·     function: string  
·     begin  
·       Result := HttpClient.Get('https://api.com/user/1');  
·     end  
·   ).ThenByAsync<TUser>(  
·     function(Json: string): TUser  
·     begin  
·       Result := TJson.JsonToObject<TUser>(Json);  
40     end  
·   ).OnComplete<TUser>(  
·     procedure(User: TUser)  
·     begin  
·       ShowMessage(User.Name);  
·     end  
·   ).OnException(  
·     procedure(E: Exception)  
·     begin  
·       ShowMessage(E.ToString);  
50     end  
·   ).Start;  
· end;
```



Muito obrigado!

www.cesarromero.com.br

github.com/cesarliws



www.linkedin.com/in/cesarliws

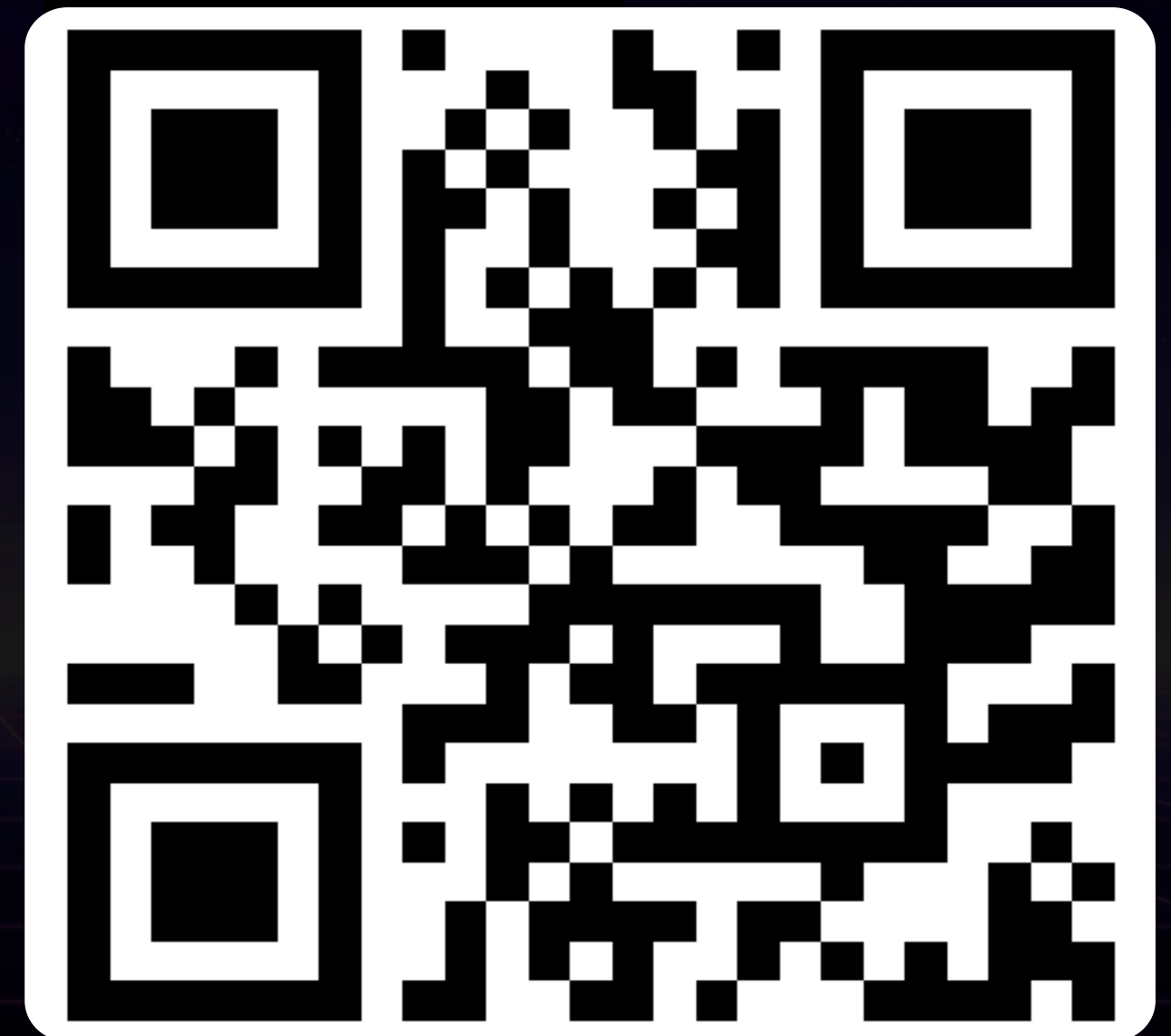


cesarliws@gmail.com



Cesar Romero

Acesse este QR Code com o
APP do Evento e
avalie esta apresentação





www.cesarromero.com.br